Group 14: Che-Del Mahinay, Visal Samarakoon, Mark Fernando

API-based Drone Simulation Development using Airsim and ArduPilot

Project Report

Multidisciplinary Innovation Project

School of Information and Communications Technology

Metropolia University of Applied Sciences

Project Report

15 May 2026

Contents

1 Introduction

Unmanned Aerial Vehicles (UAVs), commonly referred to as drones, have become an essential component of modern technological systems. Their applications span multiple domains including logistics, environmental monitoring, surveillance, and autonomous systems development. As these applications continue to expand, drone systems are increasingly required to operate with high levels of reliability, precision, and autonomy.

A modern drone is a complex system composed of multiple interacting components, including sensors, actuators, embedded controllers, and communication modules. At the core of this system lies the flight controller, which is responsible for stabilizing the drone and executing movement commands based on input data and control algorithms. The integration of these components introduces significant complexity in the design and testing of drone systems.

Developing and testing drone behavior directly on physical hardware presents several challenges. These include the risk of hardware damage, safety hazards, and limited debugging visibility. Errors in flight logic highlight the importance of simulation-based approaches in drone system development. As a solution, Open-source tools such as AirSim enable realistic 3D simulation of drone dynamics and sensor behavior, while ArduPilot provides a widely used open-source flight control system capable of operating in Software-in-the-Loop (SITL) environments.

The project aims to integrate ArduPilot and AirSim through an API, enabling drone control and monitoring within a simulation environment. During the implementation process, the team will gain experience not only in C++ and Python programming in a UAV drone sense but also in API-based control of drone functions such as arming, takeoff, waypoint navigation, return-to-launch, and sensor interaction, while producing supporting documentation for installation, configuration, and testing.

1.1 Problem Statement, Objectives, Research Questions

Developing and testing drones on physical hardware presents challenges such as hardware damage, safety risks, and limited debugging visibility. Software errors can lead to crashes, increasing development time and cost. Simulation-based environments help reduce these risks by enabling safer testing while still emulating real-world drone behavior.

The main objective of this project is to develop an API-based drone simulation environment using AirSim and ArduPilot. The goal is to create a repeatable development setup where ArduPilot SITL can be integrated with AirSim to test drone behavior in simulation. Another objective is to provide API-based implementations for drone operations such as arming, takeoff, waypoint navigation, return-to-launch, and sensor interaction, along with supporting documentation for setup, usage, and testing.

This project addresses the following main questions: how can AirSim and ArduPilot SITL be integrated into a repeatable drone simulation environment? How can API-based interaction be used to control drone movement, missions, and sensor-related behavior within the simulation? How can manual and autonomous flight be implemented in the drone? And how can this setup improve testing and development before deployment to physical drone hardware?

1.2 Scope and Limitations

Define the boundaries of the project (e.g. it includes Airsim+Ardupilot, API, simulation development, making physical drone. It's limited to timeline, focuses only on basic drone behavior etc.

Scope

The project's initial goal is the development of drone simulation using AirSim and ArduPilot. The simulation provides a flexible working environment and helps to test without the risk of damaging the drone. This offers an opportunity to experiment with different parameters.

The simulation environment is used to test basic drone behaviours such as take-off, hovering and landing. These behaviours are used to simulate the drone flying from point A to B and returning to the origin. After successful simulation, the real drone can be tested with the same setup.

Limitations

The project is limited to testing basic drone behaviours and aimed at a single drone. Advanced features such as obstacle avoidance and AI vision are not included.

1.3 Significance of the Project

Unmanned aerial vehicles are used in a wide range of real-world applications such as aerial inspection, environmental monitoring, logistics and emergency response. Due to the critical nature of these tasks, the development of safe and reliable drone systems has become increasingly important. However, testing drone software directly on physical hardware can be risky, especially during the early stages of development. As crashes caused by software bugs or erroneous flight logic may damage not only hardware, but also create safety hazards. Therefore, simulation environments have become a leading tool in modern drone development.

This project is significant because it demonstrates how a simulation-based development can be implemented using Microsoft AirSim and the ArduPilot flight control system. Developers can test drone behaviors, flight commands and mission logic within a controlled virtual environment before deploying them on real hardware by integrating these systems through an Application Programming Interface (API). This is an approach that helps reduce development costs, improve safety and enable faster iteration during the design and testing phases of drone systems.

In addition to its practical value, the project contributes to learning in several technical aspects. The project team gains hands-on experience with drone flight control systems, software-in-the-loop (SITL) simulation and API-based system

integration through the development of the simulation environment. The project also helps develop skills in programming, system configuration, debugging and testing within complex software environments. Furthermore, working with AirSim and ArduPilot provides practical exposure to tools commonly used in robotics and autonomous system development.

Overall, the project provides both practical and educational value by demonstrating how simulation technologies can support safer drone development while also strengthening technical skills related to embedded systems, robotics software and system integration.

2 Theoretical Background of UAV Systems and Simulation

2.1 Evolution and Principles of UAV

Unmanned Aerial Vehicles, commonly known as drones, have expanded significantly in recent years. From being experimental tools for research and hobbyist applications, air drones have transitioned into performing critical tasks across logistics, entertainment, emergency response, surveying, and even militarized missions.

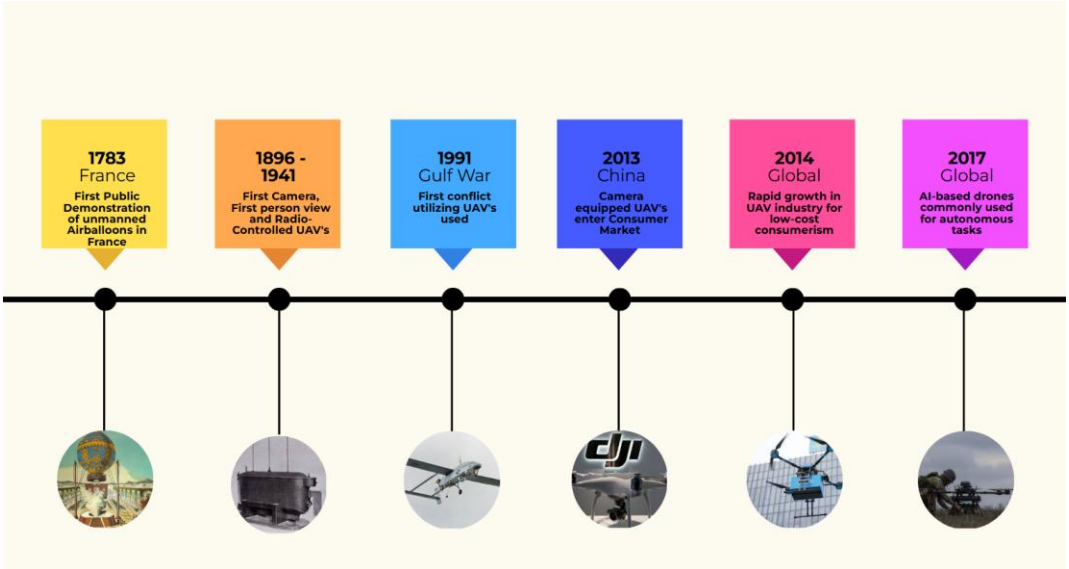

Figure 1: UAV Evolution [1]

The timeline highlights key milestones such as the introduction of radio-controlled UAVs in the early 20th century and the rapid commercialization of drones in 2010 [2]. These developments were made possible by continuous improvements in propulsion systems, control mechanisms, and embedded technologies.

Drone designs began with single propeller systems derived from early helicopter technology in the early 1900s, with unmanned versions emerging in the 1960s. Dual-propellers followed in the 1940s–1950s, improving stability and lift through coaxial or tandem designs. By the 2000s three propeller systems appeared in hobbyist applications, while quadcopters, though conceptualized as early as 1907, became the dominant design in the 2000s–2010s due to advances in sensors, microcontrollers, and flight control [3].

UAV Drones have variations based on their use cases and types. In this Project, Privately-built UAV quadcopter is the drone discussed, as it is the platform used in this project. First-person-view (FPV) quadcopters with an X-shaped body frame are also highlighted because they are adaptable to use cases and share the same components as the FPV X-frame quadcopter for this project.

UAV Drones can be commonly classified due to their weight. For instance, Military drones used by the UK armed forces are classified into three classes, with Class I further subdivided into categories (a–d) [4]. In Finland, drones are based on EU regulations, where drones are grouped by their take-off weight among under 250g, up to 4kg, and up to 25kg. [5]

Drone Category	Weight	Operational Restrictions
Privately built / Legacy (<250 g)	< 250 g	May fly close to people but not over crowds; lowest restrictions
C0 (Class-labeled drone)	< 250 g	May fly close to people; no flight over assemblies; max altitude 120 m
C1 (Class-labeled drone)	< 900 g	Limited flight over people; avoid prolonged overflight; max altitude 120 m
C2 (Class-labeled drone)	< 4 kg	Must keep distance from people (≈30 m, reducible to 5 m); max altitude 120 m
C3 / C4 (Class-labeled drone)	< 25 kg	Must be flown far from people and urban areas (≥150 m); max altitude 120 m
Privately built / Legacy (<25 kg)	< 25 kg	Restricted to far-from-people operations (A3); ≥150 m distance from people and urban areas

Figure 2: Drone classifications in Finland on weight and certification type [5]

The table classifies drones in Finland that follow standard categories. Lower classes such as C0 and C1 allow operations closer to people, while higher classes C2 to C4 have stricter restrictions. As the weight increases, operations become more restrictive due to higher risk. In contrast, legacy or privately built drones are generally subject to stricter conditions, especially in higher weight ranges. [5]

2.2 Quadcopter component overview

A modern quadcopter can be organized into three classified main systems: the Flight system, Power system, and FPV system. Flight system includes both the drone's onboard components and the ground control station components used by the operator. The onboard components handle flight and stability, while the ground control components allow the user to control and monitor the drone. The Flight system and Power system are most essential among the three component groups as they work together to allow drone flight, and its proper functionality and performance. [6]

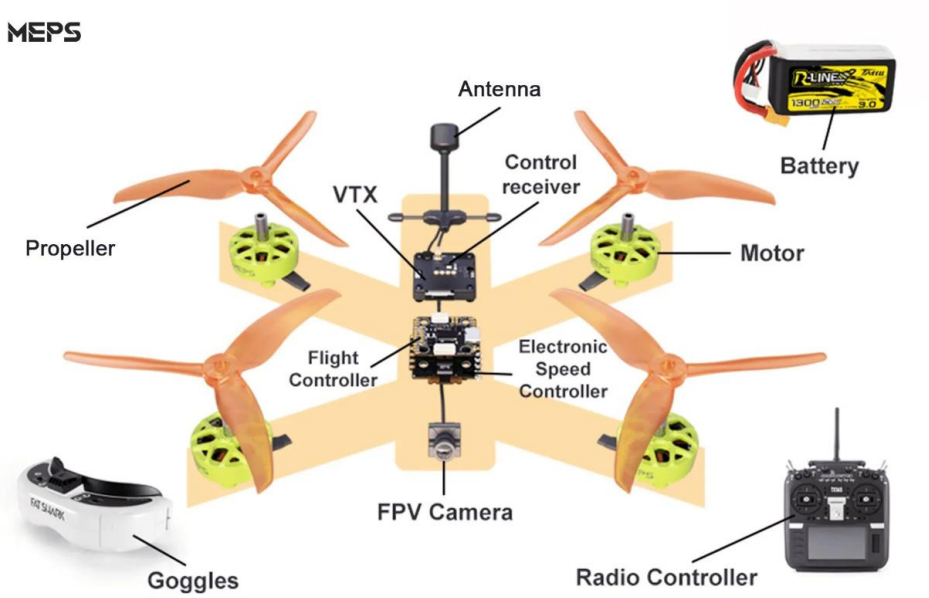

Figure 3: Modern Quadcopter Drone Parts [6]

- Flight system

  The flight system manages the quadcopter's motion, stability, and control. It includes the Frame, flight controller, receiver, and onboard sensors such as the Inertial Measurement Unit (IMU). The flight controller processes user inputs from the receiver and sensor data to maintain stability and execute flight commands. Based on these inputs, it generates control signals that determine motor behaviour. This subsystem focuses on decision-making and control rather than power delivery.

- Power system

  The power system supplies and distributes electrical energy to all onboard components. It mainly consists of a battery and a power distribution mechanism called ESC. The battery, typically a lithium-polymer (LiPo) type, provides the energy required for propulsion and control. The distribution system supplies electrical power to onboard electronics, including the ESCs and control components such as the flight controller and receiver. Its performance directly affects flight time, efficiency, and overall system reliability.

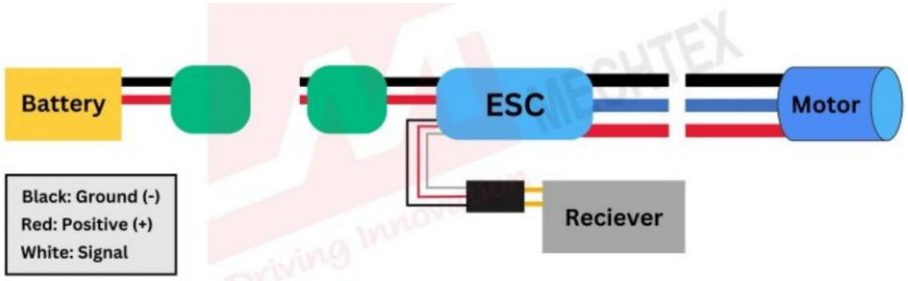

Figure 4: Electrical connection diagram of the UAV power system [7]

A UAV quadcopter's power system commonly works when flight controller sends a PWM signal to receiver connected to the ESC, indicating the desired motor speed. The ESC interprets this signal, draws power from the battery, and converts the DC input into controlled electrical pulses for the motor. By adjusting this output continuously, the ESC regulates motor speed and thrust, enabling stable and responsive flight.

- FPV system
  The FPV (First-Person View) system provides real-time visual feedback from the drone to the operator. It typically includes an FPV camera, video transmitter (VTX), and antenna, which transmit video signals to external display devices such as goggles or monitors.

Quadcopters operate either through manual or autonomous control modes. In manual mode, the user directly controls the drone using Radio Controller via radio transmitter, while in autonomous mode, the drone relies on flashed firmware program and data such as from sensors to perform tasks independently. Consequently, the selection and configuration of components are influenced by both the application requirements and the intended mode of operation.

2.4 Quadcopter Flight Dynamics

A quadcopter's movement is controlled by changing the speed of its four motors. Simply put, when all motors spin faster, the drone increases lift and moves upward, known as throttle. However, this motion also activates a twisting force from the propellers called torque, spinning the frame into opposite direction. To move forward or backward, the drone tilts using pitch by increasing the speed of

motors on one side while decreasing the other. Sideways movement is controlled through roll, which adjusts the left and right motors. Rotation around the vertical axis is called yaw and is achieved by creating a difference between clockwise and counterclockwise rotating propellers. These four controls which are throttle, pitch, roll, and yaw, allowing the drone to hover, move in different directions, and maintain stability during flight.

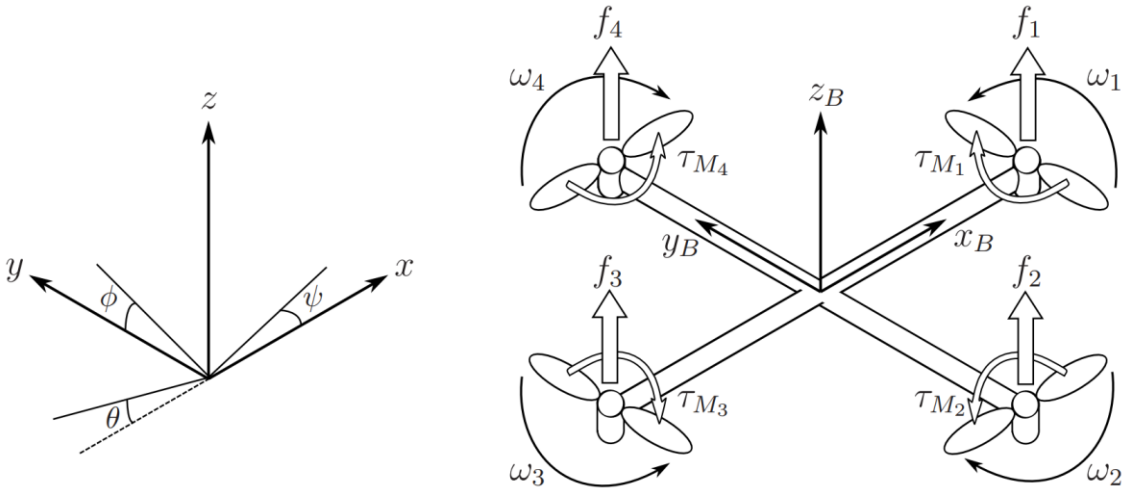

Figure 5: Inertial Body frames of a Quadcopter [9]

Referencing figure 5, the quadcopter can roll when the thrust on one side differs from the thrust on the opposite side, causing the body to tilt sideways about the x-axis:

$$T\Phi = l(f_4 - f_2)$$

The quadcopter can pitch when the thrust between the front and rear rotor pair becomes unequal, making the body tilt forward or backward about the y-axis:

$$T\theta = l(f_3 - f_1)$$

The quadcopter yaws when the total reaction torque of one propeller rotation direction becomes greater than that of the opposite direction, rotating the body about the z-axis:

$$T\psi = (TM_1 + TM_3) - (TM_2 + TM_4)$$

The quadcopter's motion is nonlinear and is within a three-dimensional coordinate system using a body-fixed (x, y, z) reference [9]. Rotational dynamics: roll, pitch, and yaw are expressed through the Euler angles ϕ, θ, and ψ, corresponding to rotations about the x, y, and z axes, respectively [9]. Also, $f\ or\ thrust\ force, T\ or\ torque\ and\ l$ or arm length from center to motor is also considered. Such logic are present in simulation environments as they capture real drone dynamics or physics.

2.5 Drone Simulation Environment system

Drone Simulation Environments accomplishes the actual virtual control of the physics and behaviour of drones and its systems. Virtual drone testing solves the problem on designing, validating reliable systems for drones without exposing it to unnecessary risks and failures. These are carried out through testing the drone's actual physics, dynamics and tests runs in computers where physical crashes due to software and human errors are impossible.

AirSim, run on Unreal Engine software, is an open-source simulation platform for drones, cars, and other autonomous systems. This platform supports programmatic interaction through its APIs and can be integrated with external flight controllers such as ArduPilot. In drone development, this makes AirSim useful for evaluating flight behaviour in three-dimensional simulated scenes while maintaining a connection to real autopilot logic.

A common simulation setup combines AirSim, Unreal Engine, ArduPilot, Ubuntu, and QGroundControl. Ardupilot running on Ubuntu via Windows Subsystem for Linux (WSL) through a Software in the Loop (SITL) enables the simulation without actually requiring real hardware. Qground Control is a software applicable to Ardupilot firmware and it is used as an interface for telemetry monitoring, navigation, and mission planning.

2.6 Drone System API and Communication Protocols

An Application Programming Interface (API) is a software interface that allows one program to communicate with and control another system using defined commands and responses. In drone systems, APIs are important because they enable developers to control a vehicle programmatically instead of relying only on manual piloting. Through APIs, external scripts or applications can send commands such as arming, takeoff, movement, landing, and mission execution, while also retrieving telemetry and vehicle state information. In simulation-based development, APIs improve repeatability, automation, and flexibility. They are especially valuable when testing mission logic, autonomous behavior, or sensor-based functions within a simulated environment.

Communication protocols define how data is exchanged between the components of a drone system. In API-based drone simulation, communication is required between the simulator, the autopilot, and any external control applications. A widely used protocol in this field is MAVLink, which is a lightweight messaging protocol designed for drones and other robotic systems. MAVLink supports the exchange of telemetry, commands, mission items, and parameter data between components. In a setup that combines AirSim and ArduPilot SITL, communication protocols make it possible for the autopilot to receive flight commands, send back state information, and interact with higher-level applications. Understanding this communication layer is essential because it forms the basis of reliable integration between the simulation environment and the flight control software.

3 Methodology

The project’s system follows core implementations in its work environment. AirSim and Unreal Engine provides the 3D simulation environment, while ArduPilot handles the flight control logic. These components are connected through DroneKit API, allowing external programs to control the drone and receive data. As expected, this project follows a simulation-first approach to safely test drone behavior before using real hardware.

The project environment was set up on both Linux and Windows systems. On Linux, all components were run directly, while on Windows, Ubuntu WSL was used to run DroneKit and MAVProxy, with AirSim running on the Windows side. This setup showed that the system works across two operating system platforms.

3.1 Project's Work Breakdown Structure

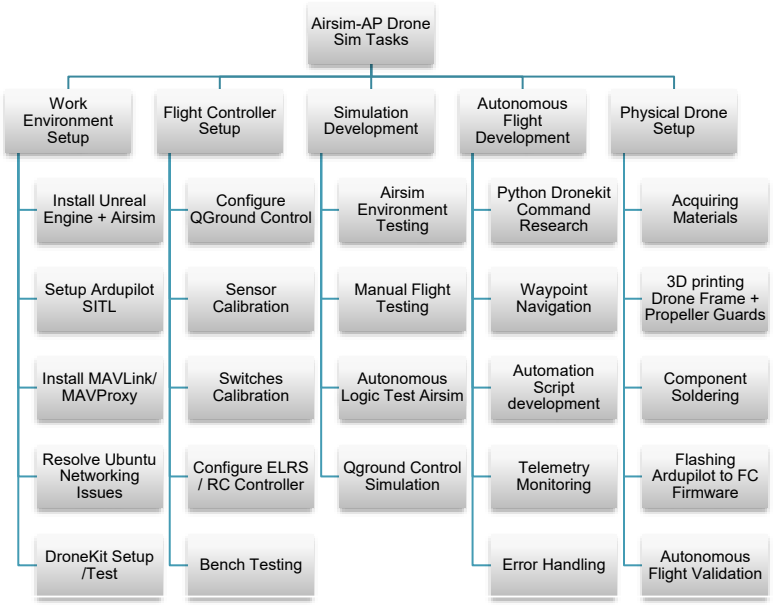

Figure 6: WBS of Drone Simulation project Tasks

Based on the display above, the Work Breakdown Structure of the project consists of Five main tasks:

- Work Environment Setup
- Flight Controller Setup
- Simulation Development
- Autonomous Flight Development
- Physical Drone Setup

Among the Five main tasks, Work environment setup is implemented first. This step installs the necessary software needed to implement and ensure the performance development of the other four main tasks. The of Unreal Engine installed in the setup is v4.27.2 strictly as it is the latest version compatible with

Airsim. Next the simulation development is carried out to begin the visual testing of the drone flight logic either manual or autonomous which also familiarises the project members how the drone would fly through each command in real life testing. Once the simulation environment is configured, simulation development is conducted to test communication, flight commands, and drone behavior safely before real-world deployment. The remaining tasks focus on autonomous flight logic, configuring the flight controller, and implementing as well as integrating the physical drone hardware. Autonomous flight behavior is implemented using waypoint missions and Python DroneKit scripts while QGroundControl is used for calibration, telemetry visualization, and flight parameter adjustments.

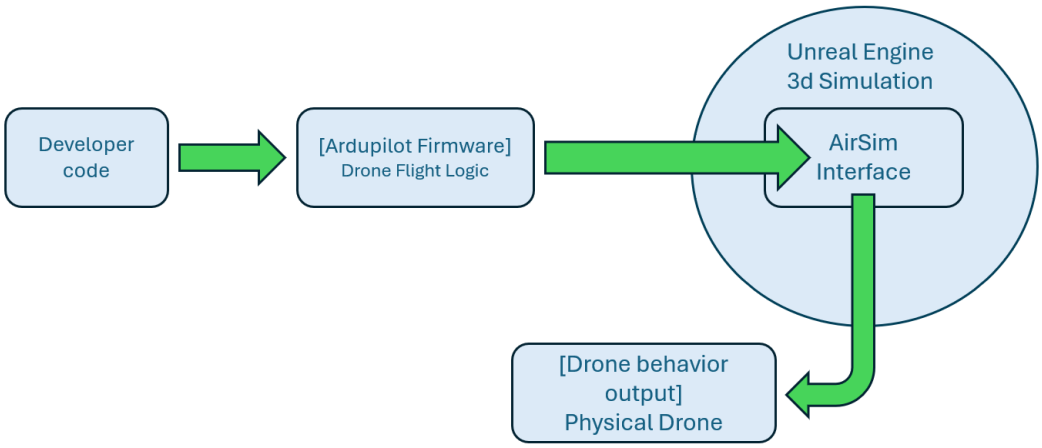

Figure 7: Airsim-ArduPilot Project Development Workflow

A broader simplified view of the WBS shows this project follows flight logic development, simulation and validation that will ultimately lead to the physical testing of the flight control system itself using a physical drone. Once the flight control logic has been developed, it will be validated through simulation to enable iterative testing of the drone movements, commands and scenario responses.

3.2 Drone Communication and Navigation

The project uses an ELRS radio system communication to establish a data link between the drone and the remote controller. The setup is designed to support sustainable manual control and autonomous flight while receiving telemetry either both on simulated or physical drone deployment.

The Drone specifically uses an ELRS Nano receiver RX connected to the flight controller to send and receive flight signals wirelessly to and from the drone system while the Controller itself already has a built-in ELRS transmitter. Proper installations of the firmware of the FC is needed before achieving communication between the Radio Transmitter and receiver component of the Drone.

Figure 8: BetaFPV Lite 3 Radio Transmitter Controller [10]

The controller that was used is a lightweight radio controller that features an SX1280 firmware ELRS protocol that connects the Flight controller and the Drone itself via the ELRS nano TX 2.4G module transmitter component of the drone. The controller already has a built-in ELRS transmitter and is designed mainly for RC models such as multi rotors which includes quadcopter used in the project.

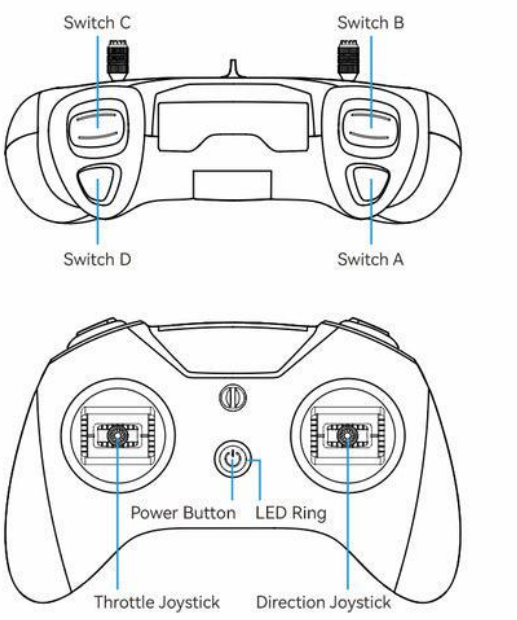

Figure 9: BetaFPV lite 2 Diagram [10]

One of the main features of the BetaFPV controller is its support compatibility for our software Drone simulation system. The controller can simulate drone flight using flight planning software such as Qground control that is applicable for ArduPilot flight logic that is used by the drone's Flight controller.

The controller is configurable by mapping its input channels to flight controls such as roll, pitch, yaw, and throttle, allowing proper interaction with the drone. The Left Gimbal represents the throttle while the Right represents the roll pitch and yaw. The switches are also configurable to the user allowing toggling between manual to autonomous flight control. Finally, calibration is performed to align input values and set correct neutral positions, ensuring accurate response and stable control during simulation.

3.3 QGroundControl

The project used QgroundControl to configure the Ardupilot controller and manage autonomous flight settings too. QgroundControl's software allowed the team to calibrate the actual drone's sensors, radio controller setup and waypoint mission through a graphical interface in the computer. Additionally, it was used for monitoring telemetry and verify whether the drone is responding correctly with the commands.

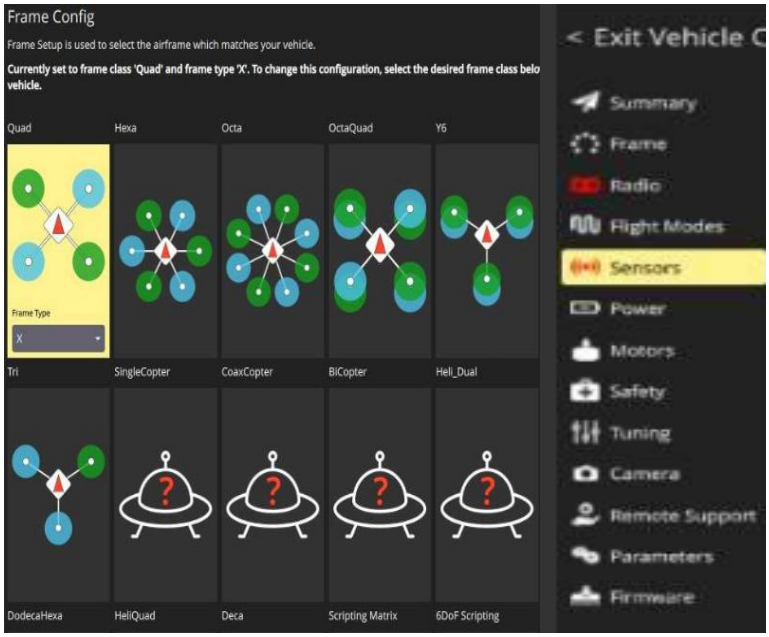

Figure 10: QGroundControl configuration options

QGroundControl was also used to configure the drone frame type, flight modes, and safety parameters before autonomous testing. Before the drone can be flown autonomously, the QgroundControl must first both setup the ardupilot firmware and configure the Qgroundcontrols in the drone's FlightController via linking it with a laptop which the software is installed.

5 Implementation

This chapter explains the key topics on how the Drone simulation system was established. It covers three main topics which are Physical Drone assembly, Drone Communication setup, and the Drone flight logic program.

5.1 Physical Drone Assembly

Based on the covered topics for the FPV drone component build, a similar component design of the quadcopter drone is similarly implemented to use components such as four motors and propellers, Flight controller, Electronic Speed Controller, controller receiver and antenna and Lithium-ion Polymer (LiPo) for the battery.

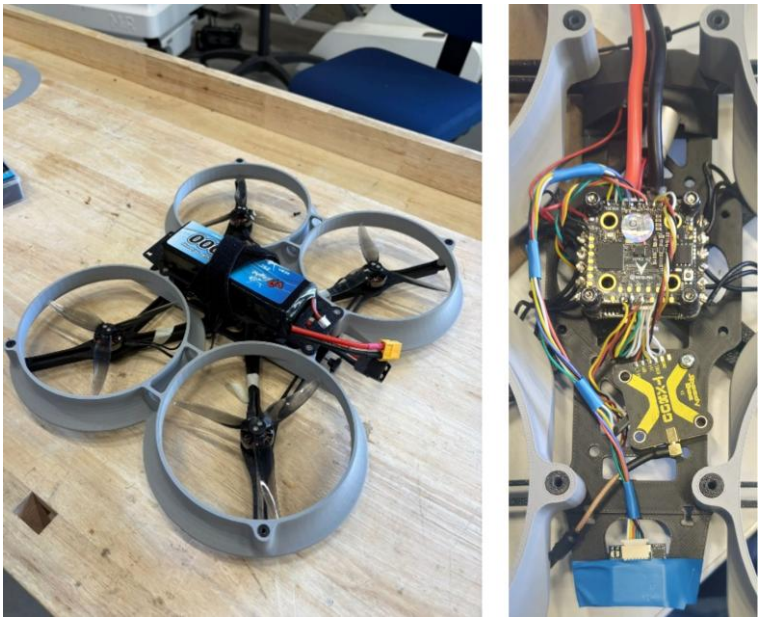

Figure 11: Flight Ready Drone Display (Left)
Stripped-down Central Drone Structure (Right)

It was revealed during the later stages of the project that a GPS component with compass was needed to fly the drone autonomously safely; therefore, it was also successfully attached to the drone. However, such FPV components namely: Camera, FTX and goggles were disregarded from the system as First person viewing is deemed unnecessary to the simulation system goal of the project.

5.1.2 Drone Wiring Configurations

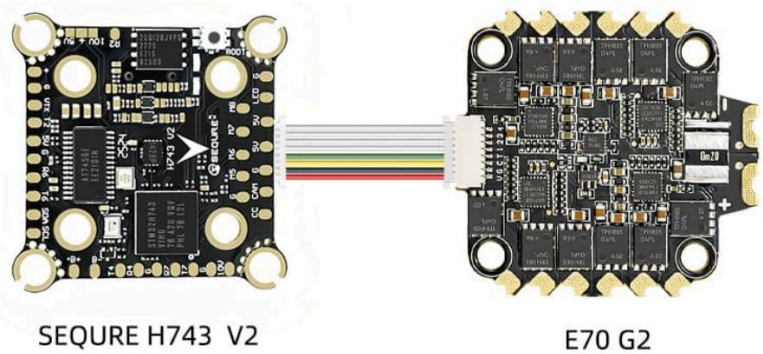

Figure 12: ESC to FC Component Wiring Link

The actual drone platform used a SEQURE H743 V2 flight controller with an E70 G2 70A 4-in-1 ESC. Both components are connected using a plug-in cable

connector. The two components are assembled so that the Flight controller is on top of the ESC, supported by four pillars on each corner.

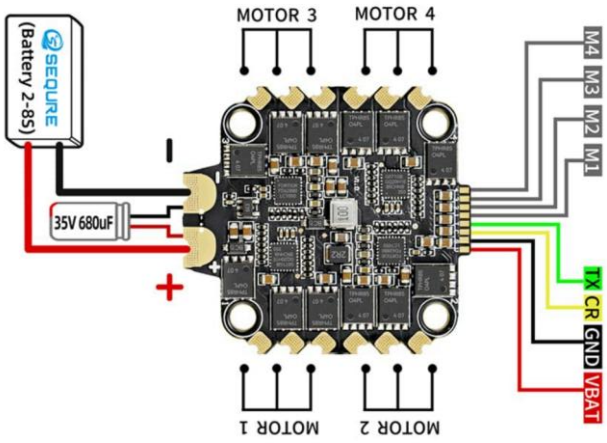

Figure 15: Electrical Speed Controller (ESC) Diagram

Referring to figure 15, the wiring design that was implemented to the ESC was similar to the actual product's wiring diagram. Wires from motors 1 – 4 were soldered to the ESC's motor solder pads respectively so that it receives motor control signals from the flight controller through the four motor communication lines.

The GND and VCC wire of the LiPo battery was also soldered to the ESC's respective solder pads to convert battery power into regulated outputs for the four drone motors. Lastly, 35V capacitor was soldered to the ESC's solder pad to minimize electrical noise, improving power stability.

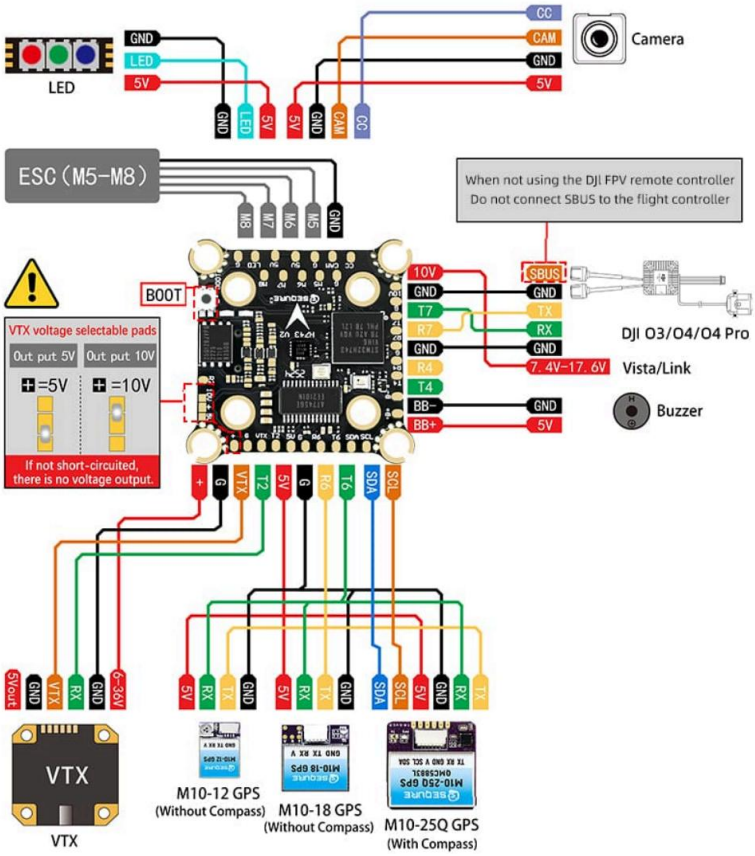

Figure 13: Sequre H743 V2 Pad Wiring Diagram

The above figure displays the FC's upper surface that follows our combined UART and I2C wiring for the M10-182Q GPS. Its QMC5883L compass was connected using UART communication for GPS positioning data and I2C communication was used through the SDA and SCL pads. As previously stated, The VTX and Camera was omitted in this project, however its solder pads were repurposed in order for the The communication structure of ELRS flight receiver is shown as UART to be attached to the FC.

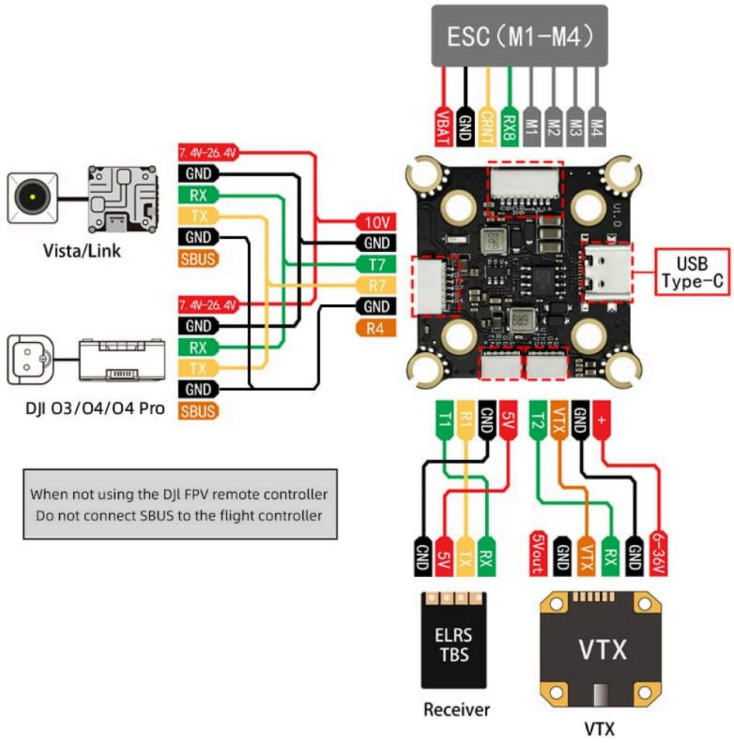

Figure 14: Sequre H743 V2 Socket Wiring Diagram

The above figure displays the FC's bottom surface of the FC. The USB type-C cable acts as the gateway link to enable the drone's firmware flashing, ArduPilot installation, parameter configuration, telemetry access, and communication between the flight controller and QGroundControl during testing and debugging procedures.

Both figure 13 and 14 additionally show the ESC communication, however their difference comes to which they are connected with and how each surfaces achieve connection. In this project the lower surface is connected to the ESC using a UART plug-in cable as shown in the Figure 12 diagram, enabling connection and motor operation to motors 1-4 of the project's quadcopter.

5.2 Main Program

Autonomous Python Scripts were developed and tested for initially under simulation and eventually for drone deployment in this project. Generally, the python scripts use Dronekit API that uses MAVLink communication to send commands to an Ardupilot flight logic both in SITL simulations and Actual Drone deployments.

5.2.1 Take-off

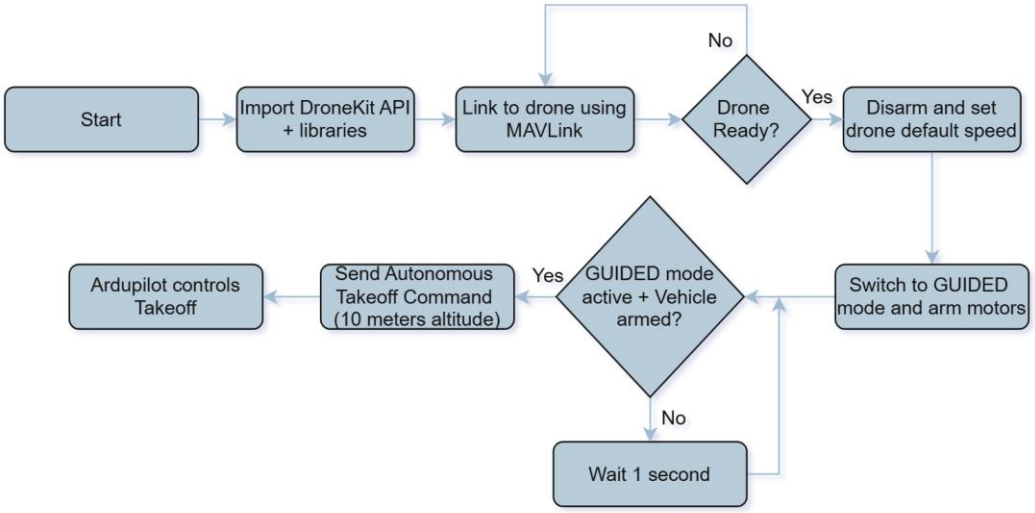

Figure 15: Take-off Python Script Flowchart

The takeoff script first connects to the drone vehicle using the DroneKit API through MAVLink communication. After linking, the script changes the drone into GUIDED mode and arms the motors so it can accept autonomous commands. Once the drone is ready, the script sends a takeoff command with a target altitude. ArduPilot then handles the flight behavior either in the simulation environment or during actual drone testing.

5.2.3 Move the Drone from A-B

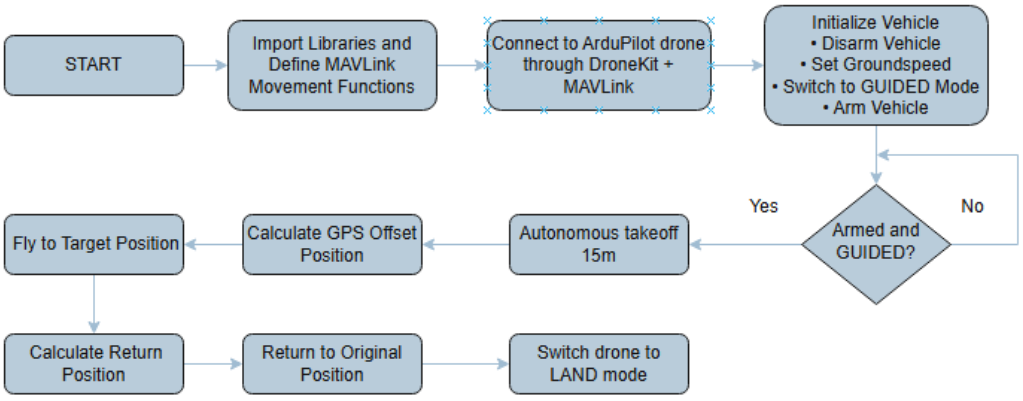

Figure 16: A-B drone flight plan script Flowchart

Th A-B flight plan script, as with the other two scripts, first imports the libraries and define MAVLink functions. Then it connects to the drone simulation

environment, arms the drone, and switches it into GUIDED mode for autonomous control. After takeoff, the program calculates new GPS target positions relative to the drone's current location and commands the drone to move toward those coordinates automatically. The drone then returns to its original position before finally switching into LAND mode.

5.2.3 Land

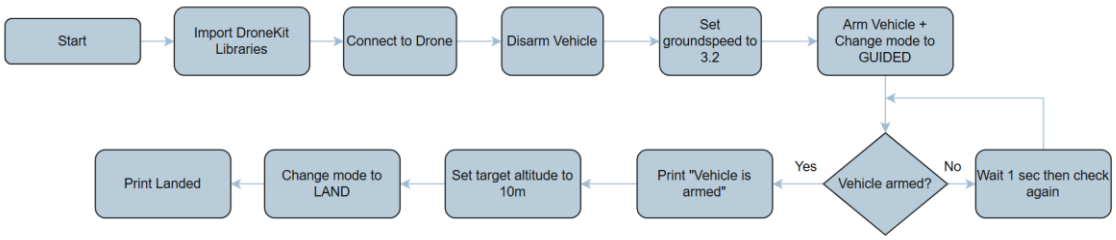

Figure 17: Land Python Script Flowchart

Lastly, the Land script imports libraries, then connects the drone. Then it disarms the drone and sets a default groundspeed value for movement commands. The drone is then switched into GUIDED mode and armed so it can receive autonomous commands. Once the vehicle is ready, the script prepares a target altitude value for takeoff. However, instead of continuing with the takeoff command, the script changes the drone mode into LAND mode, which commands the drone to land immediately.

4 Results

The project successfully demonstrated autonomous drone simulations using AirSim and ArduPilot together with MAVLink communication and DroneKit scripting. The simulation environment was able to execute autonomous commands such as takeoff, waypoint movement, and landing. The availability of drone components inside the RoboGarage laboratory, including the battery, flight controller, ESC, and soldering tools, helped the team assemble and configure the drone hardware more efficiently.

Products/Components	Cost in Euro (~some estimated)
SEQURE H743 V2 + E70 G2 FC/ESC Stack	118[11]
M10-182Q GPS + QMC5883L Compass	25[12]
BETAFPV ELRS Nano Receiver	9.99[13]
BETAFPV LiteRadio 3 Transmitter	32 [10]
Motors (4 pcs)	~20
Propellers (2 sets)	~40
LiPo Battery	30
3D Printed Drone Frame Materials	~25
3D Printed Propeller Guards	20
Estimated Total	319.99

Figure 18: Aggregate Cost of Project UAV Physical Drone

The developed autonomous drone platform and simulation environment resulted a cost-effective implementation of embedded UAV research and testing. The software infrastructure including ArduPilot flight firmware, MAVLink communication and Airsim with Unreal Engine uses open-source freely distributed development frameworks. This eliminates the licensing and costs commonly associated with software systems. For the component price costs, contemporary store pricing and economic status affect several component's pricing when chosen and demands continuous research.

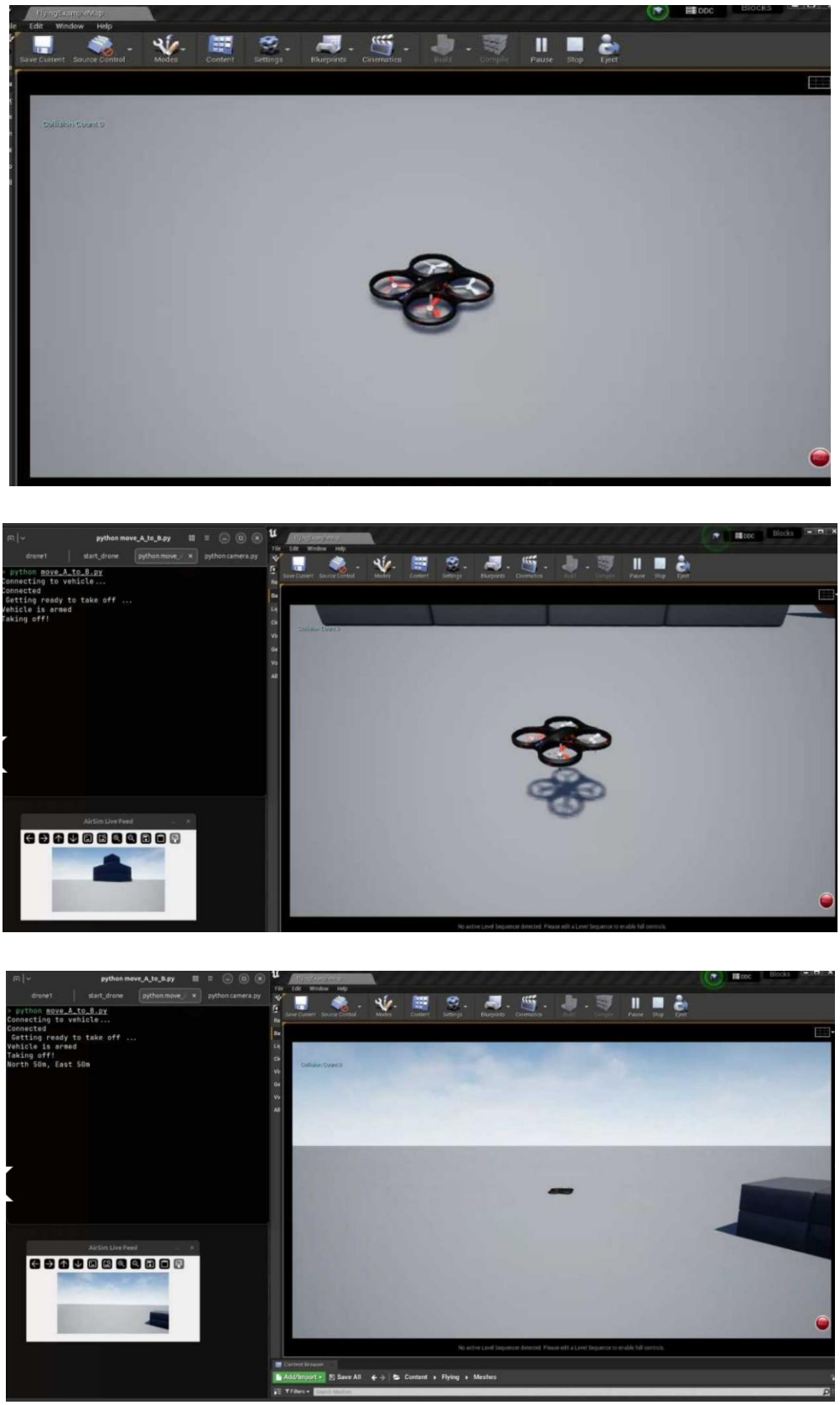

Figure 19: Sample Result: Flight-Plan Move A-B Script Execution

During testing, several factors were compared between direct physical drone deployment and simulation-first testing. These comparisons were based on repeated simulation runs together with observations from manual physical drone flights that followed similar movement behaviors from the DroneKit autonomous scripts, including takeoff altitude, movement distance, landing, and flight modes.

Results showed that direct drone deployment produced higher failure costs, while simulation testing provided easier debugging, more successful repeated tests per hour, and faster recovery time after failed tests.

Overall, the project demonstrated that performing simulations before actual drone deployment greatly improves testing safety, debugging efficiency, and development workflow reliability.

5 Limitations and Improvements

5.1 Software Limitations and Improvements

One limitation in the project timeline was due to the difficult environment setup of the project. The simulated environment used circulated around the use of Airsim, however Windows has already stopped its support already last several years ago. this led to software incompatibility issues which are either visible but difficult to fix due to varying parameters such as networking the Airsim UDP and IP address from Ardupilot to successfully run the pipeline connectivity. The solutions were found to be unofficial and heavily dependent on the solver’s configurations of the simulation system dependencies and their versions which makes it difficult if not followed similarly, leading to several risk-applied custom solutions.

The processing capacity of the device running the simulating system has also limited the processing capacity of the project’s simulation device hardware which needed enough memory for running 3-dimension simulations via Unreal engine in Airsim and communicate synchronously through time time-bound connection flight controller. The simulation workflow for the project which includes running WSL or Windows shared Linux for operating MAVLink and Ardupilot required

processing power while simultaneously running with Windows Airsim under Unreal Engine. Desynchronization due to lack of processing capacity have led to MAVLink commands failing due to unsecured GPS timing, thus disabling the drone simulation to fly at all using API and python autonomous scripts. A workaround was formulated which included setting up and running the simulation through a linux based computer running on ubuntu which successfully proved the API and autonomous scripts to finally work and be tested.

Another software limitation encountered during the project was the compatibility between the flight controller, ArduPilot firmware, and the autonomous flight configuration. After flashing ArduPilot into the flight controller, the drone motors did not follow the expected motor orientation required by Mission Planner. Two motors rotated in the wrong direction, causing thrust errors and crashes during testing. To resolve, the motors needed manual resoldering and rewiring. Another solution implemented was ESC configuration software to reverse the motor direction more safely through software adjustment.

Future improvements may include using computers with better performance capacity, implementing working and well-supported simulation system platforms. Additional automation setup such as implementing Docker may also help reduce the complexity of the configuration setups.

5.2 Hardware Limitations and Improvements

Another hardware issue was the need for a GPS component in the quadcopter drone. A key component GPS + compass which is an additional key component implemented during the middle stages of the project was applied to the physical drone system. Although autonomous flight would still be achievable with the default build alone, the GPS component allows the full capabilities of the QgroundControl, enabling the activation of drone FC and ESC sensors and to allow visibility of the Drone’s origin through its compass. Without the GPS component, it was revealed that the drone would either need a microcontroller API transmitter, likely a raspberry pi (MCU) attached to the drone or to toggle

from manual that flies the drone in a designated altitude and then switch to autonomous by that point.

Future improvements of the project may include obstacle detection sensors, LiDAR systems and computer vision technologies to improve autonomous navigation and environmental awareness.

5 Conclusion

The main goal of this Innovation Project was to design and implement a safer drone testing system using simulation before real-world deployment. The project combined AirSim simulation, ArduPilot firmware, DroneKit Python APIs, MAVLink communication, and QGroundcontrol as an interface of the physical drone platform into a single autonomous drone development environment. Through this integration, the project addressed challenges related to autonomous flight testing, communication setup, and safer drone validation.

One of the main contributions of the project is the implementation of a simulation-first testing workflow that supports both manual and autonomous drone flight. The developed system allowed waypoint navigation, telemetry monitoring, and autonomous flight logic to be tested safely inside the AirSim simulation environment before applying the commands to the real drone. The project also successfully integrated the SEQURE H743 V2 flight controller, E70 G2 ESC stack, ELRS communication system, and the M10-182Q GPS module with QMC5883L compass for autonomous drone operation.

The results of the project showed that simulation-assisted testing improved debugging safety, repeatability, and testing flexibility compared to direct hardware-only testing. The simulation environment reduced hardware risks during development while allowing repeated autonomous flight validation without exposing the drone to unnecessary crash conditions. Successful manual and autonomous flight tests also validated the communication and autonomous systems implemented in the project.

The project also demonstrated how simulation software and commercially available drone hardware can be combined to create a safer and more flexible drone development environment for educational and research purposes. Future improvements may include obstacle avoidance systems, computer vision integration, LiDAR sensors, and more advanced autonomous flight behavior.

In conclusion, the project successfully demonstrated that simulation-assisted drone development can improve the safety and flexibility of autonomous drone testing before physical deployment. The successful implementation of manual and autonomous flight behavior simulations validated the effectiveness of integrating AirSim, ArduPilot, DroneKit, ELRS communication, and GPS navigation into a complete drone simulation and testing system.

6 References

1 Boyd JE. Artificial clouds and inflammable air: The science and spectacle of the first balloon flights, 1783 [Internet]. Science History Institute. 2009 Jun 24 [cited 2026 Apr 16]. Available from: https://www.sciencehistory.org/stories/magazine/artificial-clouds-and-inflammable-air-the-science-and-spectacle-of-the-first-balloon-flights-1783/

2 Imperial War Museums. A brief history of drones [Internet]. IWM. [cited 2026 Apr 16]. Available from: https://www.iwm.org.uk/history/a-brief-history-of-drones

3 Attard D. The history of drones: a wonderful, fascinating story over 235+ years. DronesBuy.net. 2024 Mar 5 [cited 2026 Apr 19]. Available from: https://www.dronesbuy.net/history-of-drones/

4 L. Brooke-Holland, *Unmanned Aerial Vehicles (UAVs): An Introduction* [Internet]. UK Parliament, 2012 [cited 2026 Apr 19]. Available from: https://www.files.ethz.ch/isn/157096/SN06493.pdf

5 Drone Laws. Drone laws in Finland [Internet]. [cited 2026 Apr 19]. Available from: https://drone-laws.com/drone-laws-in-finland/

6 Mepsking. *What are the parts of FPV drone?* [Internet]. [cited 2026 Apr 20]. Available from: https://www.mepsking.shop/blog/what-are-the-parts-of-fpv-drone.html

7 MechTex. A comprehensive guide to electronic speed controllers [Internet]. [cited 2026 Apr 20]. Available from: https://mechtex.com/blog/a-comprehensive-guide-to-electronic-speed-controllers

8 T. Luukkonen, *"Modelling and Control of Quadcopter,"* Aalto University, School of Science, Espoo, Finland, Aug. 2011 [Internet]. [cited 2026 Apr 21]. Available: sal.aalto.fi/publications/pdf-files/eluu11_public.pdf

9 M. A. Al-Jarrah, A. Al-Rousan, and A. Al-Husban, *Simulation of the quadcopter dynamics with LQR-based control* [Internet]. ResearchGate, 2020. [cited 2026 Apr 22]. Available from: https://www.researchgate.net/publication/341657332_Simulation_of_the_Quadcopter_Dynamics_with_LQR_based_Control

10 BETAFPV, "LiteRadio 3 SIM Controller," BETAFPV, accessed May 6, 2026. [Online]. Available: BETAFPV LiteRadio 2 SIM Controller

11 SEQURE H743 V2 + E70 G2 Stack 4-8S FPV," Sequre. [Online]. Available: https://sequremall.com/products/sequre-h743-v2-e70-g2-stack-4-8s-fpv. [Accessed: May 6, 2026].

12 M10-182Q GPS Small Size Fast Positioning + QMC5883L Compass," Vertical Hobby (accessed May 15, 2026).

13 ELRS Nano Receiver," BETAFPV, accessed May 15, 2026. [Online]. https://betafpv.com/products/elrs-nano-receiver